Shipping Your Project

Autumn term 2026

Practical advice, not rules. The binding rules are in the Requirements document; nothing here changes them. If the two ever disagree, the Requirements win.

"Shipping" means handing over work that someone else can open, read and run without asking you a single question. Most marks lost on this project are lost here — not in the modelling.

Two things to do in the first week

Put your real name on your GitHub account. Requirement 4.7 asks for your full name in the account's profile name, and Requirement 4.8 says a submission that cannot be matched to a registered student is not graded. Most student accounts are called something else. Changing it takes ten seconds in Settings → Public profile, and it is the single cheapest way to avoid losing a grade you have earned.

Make the repository now, not in December. An empty repository with a README costs nothing and means the last week is about the work.

The test that matters

Before you submit, do this on a machine that is not the one you worked on:

1. `git clone` your own repository into an empty folder.
2. Follow your own README exactly as written, changing nothing.
3. See whether the results in your paper appear.

If step 3 fails, you have found what the marker would have found. Almost every problem below is caught by this one test. Cloning is the honest version of the test: it shows you only what you actually committed, not what happens to be sitting on your desk.

What the repository holds

```
paper.pdf              the research paper
README.md              how to reproduce, in order
code/                  the scripts and notebooks
data/                  the data, or how to get it
figures/               what the paper shows
recording.mp4          or a link in the README, if the file is large
```

Use lowercase names with no spaces and no accents. `data final (2).csv` breaks on other people's machines; `prices_2020_2024.csv` does not.

Working with the repository

- **Commit as you go.** A commit a day is a record of your work. One enormous commit on the last evening is a record of nothing, and it is also when things go wrong.
- **What counts is the last commit before the deadline** (Requirement 4.12). Committing is not the same as pushing: run `git push` and then look at the repository page in your browser to confirm your work is actually there.
- **Do not commit very large files.** GitHub rejects anything over 100 MB and gets slow well before that. If your data is large, Requirement 4.9 lets you commit the download script instead.
- **Use a `.gitignore`** for caches, checkpoints and virtual environments (`__pycache__/`, `.ipynb_checkpoints/`, `.venv/`). They are noise, and they make the repository heavy.
- **Never commit a key or a password.** Deleting it in a later commit does not remove it: it stays in the history. If it happens, tell us and revoke the key.

The README

Half a page is plenty. In this order:

- what the project does, in two sentences;
- what to install (`pip install -r requirements.txt` is the whole answer, if you write the requirements file);
- what to run, in the order to run it;
- how long it takes, and what it produces;
- where the data came from.

Write it as instructions to a stranger, because that is who reads it.

Code

- **A script that runs top to bottom beats clever code that needs explaining.** You are graded on evidence that you can apply the methods, not on style.
- **Set the random seed.** If your numbers change every run, nobody can check anything in your paper.
- **No absolute paths.** `/Users/anna/Desktop/thesis/data.csv` exists on exactly one computer in the world. Use `data/prices.csv`.
- **Restart the notebook and run it once, in order, before submitting.** Notebooks that work only if cells are run out of order are the most common reproducibility failure there is.
- **Keep secrets out.** No API keys in the code. If your project needs one, say in the README which key is needed and where to put it.

Data

- Small and shareable: include it.
- Large or licensed: include the download script and say plainly what the licence permits. Requirement 4.6 allows this.
- Say where every data set came from and what you did to clean it. "I dropped 412 rows with missing prices" is a sentence the marker wants to read; silence about it is not.

The paper

- Use the SIAM template from the start. Reformatting at the end always costs more than it saves.
- Every figure needs axis labels, units, and a caption saying what to look at.
- Report what you actually ran. If a method failed, that belongs in the paper — Requirement 6.3 lets an honest negative result score full marks.

The recording

- Fifteen minutes, screen plus voice. A Zoom recording of yourself is fine.
- A structure that fits the time: the question (2 min), the data (2 min), the method (4 min), the results (5 min), what you would do next (2 min).
- Watch the first minute back before submitting. Inaudible sound is the usual failure, and it cannot be fixed after the deadline.
- Say your name at the start.

Declaring your tools

Requirement 5 asks which tools you used and what for. A few honest lines:

> ChatGPT: drafted the plotting code in `figures.py`; rewrote three paragraphs of the introduction for clarity. All statistical results are my own work and were checked by hand.

Nobody is penalised for using assistants — the course teaches you to. The penalty is for not declaring them, and for being unable to explain what you handed in.

The week before the deadline

- Run the reproduction test above. Now, not on the last evening.
- Check the page count against Requirement 4.2.
- Check the eight required sections are present and in order.
- Submit a first version early. You can replace it; you cannot create it after the deadline.